

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет информатики, математики и компьютерных наук

Цапаев Данил Вадимович

**СОЗДАНИЕ ПЛАТФОРМЫ ДЛЯ АНАЛИТИКИ
ФИНАНСОВЫХ РЕЗУЛЬТАТОВ ПУБЛИЧНЫХ КОМПАНИЙ**

Выпускная квалификационная работа - БАКАЛАВРСКАЯ

РАБОТА по направлению подготовки Программная
инженерия

образовательная программа

«09.03.04 Программная инженерия»

Рецензент
приглашенный преподаватель

Е.А. Лупанова

Руководитель
старший преподаватель

Н. С.
Окуньков

Нижний Новгород, 2025

Содержание

СОЗДАНИЕ ПЛАТФОРМЫ ДЛЯ АНАЛИТИКИ ФИНАНСОВЫХ РЕЗУЛЬТАТОВ ПУБЛИЧНЫХ КОМПАНИЙ	1
Введение	2
Глава 1. Постановка задачи и разработка требований	3
1.1. Задачи и цели	4
1.2. Требования к подсистеме	5
1.3. Требования к дизайн-макетам	5
1.5. Нефункциональные требования	8
1.6. Функциональные требования:	8
Глава 2. Обзор существующих решений	10
Глава 3. Выбор инструментальных средств и технологий	15
Глава 4. Реализация	19
Глава 5. Методика и результаты тестирования	35
Глава 6. Пути возможного развития	39
Заключение	40
Ссылки на работу	42
Библиографический список	43

Введение

В связи с цифровизацией экономики и увеличением финансового массива информации повышается роль средств получения, обработки и анализа информации о финансовом состоянии публичных организаций. Инвесторам,

специалистам по инвестициям и частному сектору необходимо не только знание основных показателей отчетности, но и анализ этих данных в компактной визуальной форме. Обычно поиск информации по разным источникам занимает много времени, а получаемые данные фрагментарны и не позволяют оценить реальную финансовую ситуацию предприятия.

Исходя из этого, создание веб-сайта для анализа показателей публичных предприятий – актуальное и практически нужное дело. Система позволит интегрировать основные метрики отчётности, биржевые котировки и способы визуализации в единую среду. Таким образом, пользователь имеет возможность сравнивать показатели роста выручки, прибыли, рентабельность, уровень долгов и тд. Для принятия эффективных решений в области инвестиций.

Ещё одна важная составляющая – функционал отслеживания сделок, совершаемых пользователем через биржу. Благодаря ему пользователь сможет оценить не только результаты работы эмитента, но и сравнить свои выводы с реальными сделками. Платформа становится не информационной базой, а рабочим инструментом по ведению учёта собственного портфеля, регистрации операций и анализа доходности инвестиций.

Этим проектом я решаю конкретную задачу: создаю инструмент, который поможет анализировать эмитентов и следить за инвестиционной активностью в режиме реального времени. Платформа упростит доступ к финансовой аналитике и избавит от рутины, позволяя быстрее обрабатывать данные и принимать взвешенные решения на основе четкой структуры, а не интуиции.

Глава 1. Постановка задачи и разработка требований

1.1. Задачи и цели

Я разрабатываю веб-платформу, которая станет единым рабочим пространством для инвестора. Ее главная задача — помочь пользователю не просто смотреть на цифры, а глубоко анализировать финансовое состояние публичных компаний и одновременно вести учет собственных сделок. Система объединит в себе два ключевых направления: наглядную аналитику (выручка, прибыль, рентабельность, долги и дивиденды) и персональный трекер портфеля, где можно фиксировать операции и отслеживать доходность.

Чтобы проект превратился в полноценный рабочий инструмент, мне предстоит пройти через несколько этапов:

Сначала я изучу предметную область: разберусь, какие именно показатели критически важны для инвесторов и как лучше структурировать данные о биржевых сделках. На основе этого я сформулирую четкие требования к системе — от поиска компаний и построения графиков до автоматического расчета доходности портфеля.

Затем начнется техническая часть. Я спроектирую архитектуру приложения и создам надежную базу данных, которая сможет быстро обрабатывать большие объемы информации о рыночных данных и истории сделок.

Разработка будет разделена на два больших модуля. Первый — аналитический: он будет превращать сухие отчеты в понятные графики и расчетные коэффициенты. Второй — учетный: здесь пользователь сможет вносить свои покупки и продажи, видеть среднюю цену активов и понимать, сколько он заработал или потерял на конкретной сделке или всем портфелем.

Особое внимание я уделю интерфейсу. Он должен быть интуитивным, чтобы поиск данных и ввод сделок не превращались в рутину. После написания кода я проведу тщательное тестирование: проверю точность всех расчетов, работу фильтров и устойчивость системы к ошибкам.

В финале я оценю, насколько платформа справляется со своей задачей и куда ее развивать дальше. В будущем это могут быть автоматические загрузки данных или прямая интеграция с брокерами. В итоге получится готовый продукт, который упростит анализ рынка и сделает контроль за инвестициями прозрачным и удобным.

1.2. Требования к подсистеме

Фундаментальным этапом в выполнении работы стал процесс разработки требований. Этот шаг позволяет спланировать работу, оценить объем разработки и определить функционал готового продукта.

1.3. Требования к дизайн-макетам

При разработке дизайн-макетов платформы были сформированы требования, направленные на обеспечение удобства использования, визуальной целостности интерфейса и корректного отображения системы на различных устройствах.

- **Понятная структура и удобство навигации**

Хороший дизайн — это прежде всего удобство. Пользователь не должен блуждать по интерфейсу: основные функции, такие как информация о компании, финансовые графики или учет сделок, должны быть под рукой с первых секунд.

Старайтесь делать навигацию максимально плоской. Главные задачи лучше всего решать за пару кликов, не заставляя человека проваливаться вглубь меню. Сложную структуру можно оставить только для служебных или административных разделов, где спешка не так критична.

Когда человек добавляет сделку, сохраняет данные или допускает ошибку, он должен сразу видеть результат. Удобно, если в верхней части экрана всплывают короткие уведомления: зеленые для успеха, желтые для предупреждений и красные для ошибок. Это избавляет от лишних вопросов и делает работу с системой интуитивной.

- **Единый визуальный стиль платформы**

Платформа должна выглядеть как единое целое: стиль должен сразу считываться как инструмент для финансовой аналитики и инвестиционного мониторинга. Пользователь, открывая интерфейс, должен почувствовать надежность и технологичность продукта. Это значит, что всё — от шрифтов и кнопок до сложных таблиц и графиков — должно работать в рамках одной стройной дизайн-системы.

Визуальный стиль, включая логотип, должен транслировать суть сервиса: работу с рыночными данными и инвестиционными расчетами. Чтобы подчеркнуть деловой характер системы, лучше придерживаться палитры, которая ассоциируется со стабильностью и цифровой средой. Хорошо подойдут глубокие оттенки синего, темно-зеленого или графитового — это проверенные цвета, которые вызывают доверие у профессионалов.

- **Адаптация интерфейса под различные устройства**

Макеты должны работать везде: и на маленьком смартфоне, и на огромном мониторе. Главное, чтобы интерфейс оставался читаемым и удобным, какой бы экран пользователь ни

выбрал. При проектировании мы закладываем как минимум четыре контрольные точки: для компактных телефонов, стандартных мобильных устройств, планшетов и широких десктопов. Но дело не только в том, чтобы просто сжать или растянуть блоки. Нужно продумать саму логику: как перестроится меню, как изменятся таблицы, карточки или графики. Адаптивность — это не просто масштабирование, а умная перестройка контента под доступное пространство.

- **Использование компонентного подхода в дизайне**

Чтобы поддерживать интерфейс в едином стиле, нужно собрать повторяющиеся элементы в библиотеку переиспользуемых компонентов. Это могут быть кнопки, поля ввода, карточки компаний, таблицы, модальные окна, навигация, уведомления, фильтры и графики.

Важно продумать каждый компонент до мелочей: как он выглядит в обычном состоянии, при наведении, в момент загрузки или если произошла ошибка. Если заранее предусмотреть все варианты — активный, отключенный или ошибочный — интерфейс станет предсказуемым, а масштабировать платформу будет гораздо проще. Главная цель здесь — создать универсальные инструменты, которые можно просто брать и вставлять в любой раздел системы, не тратя время на отрисовку новых решений.

- **Информативность экранов и логика взаимодействия**

На каждом экране должно быть ровно столько информации, сколько нужно для дела. Не стоит перегружать интерфейс лишним шумом, но и заставлять пользователя искать пояснения где-то на стороне — плохая идея. Все, что требуется для анализа финансов или контроля сделок, должно быть под рукой, прямо внутри платформы.

Важна и сама логика: человек должен с первого взгляда понимать, куда нажать, как отфильтровать данные, переключить период или открыть детали по компании. Чем меньше пользователю приходится задумываться над тем, как работает система, тем проще и полезнее она становится.

1.4. Требования к интерфейсной части веб-приложения

К интерфейсной части веб-приложения предъявляется ряд требований, обеспечивающих удобство разработки, поддерживаемость проекта, качество пользовательского взаимодействия и стабильность работы системы в целом.

- **Архитектура клиентской части**

Интерфейс лучше строить на компонентном подходе. Так мы разделим логику на понятные уровни: от простых базовых элементов до полноценных страниц. Каждый компонент будет отвечать за свою задачу, а нам это даст возможность легко переиспользовать код и масштабировать проект, не превращая его в хаос.

С папками тоже стоит не экспериментировать, а придерживаться строгой логики. Разделим всё по зонам ответственности — так разработчикам будет проще ориентироваться, когда проект разрастется. То же самое касается и маршрутов: URL должны быть предсказуемыми и отражать суть страниц. Если пользователь переходит в аналитику компаний, его карточку эмитента или историю сделок, адрес в строке браузера должен быть максимально понятным и логичным.

- **Типизация и контроль данных**

Клиентскую часть пишем на TypeScript. Строгая типизация здесь — способ избежать ошибок и сделать код понятным при поддержке. Типизировать нужно всё: компоненты, утилиты, сервисы, стейт и аргументы функций. Отдельное внимание уделим данным из API. Поскольку мы работаем с финансами, любая неточность в структуре запросов или ответах сервера критична, поэтому проверка входящих и исходящих данных должна быть максимально жесткой.

- **Маршрутизация и разграничение доступа**

Нужно настроить проверку прав доступа так, чтобы система перехватывала пользователя при попытке перехода на закрытые страницы. Если человек пытается зайти в личный кабинет или админку, приложение должно сначала проверить его токен и роль через промежуточный слой маршрутов. Это база для защиты данных: истории сделок, настроек профиля и пользовательского портфеля. При этом не стоит усложнять жизнь пользователю — общую аналитику и справочные разделы можно оставить открытыми, а вот персональную информацию нужно держать под замком.

- **Управление состоянием приложения**

Если данные нужны в разных частях интерфейса, лучше не плодить копии, а использовать центральное хранилище. В него стоит выносить всё, что должно быть под рукой: профиль пользователя, настройки приложения и ответы от API, которые запрашивают сразу несколько модулей.

С бизнес-логикой та же история: не стоит размазывать её по всему коду. Лучше собрать повторяющиеся операции в отдельные функции или независимые модули. Так проект станет чище, код перестанет дублироваться, а компоненты не будут слишком сильно зависеть друг от друга. В итоге поддерживать и развивать приложение будет гораздо проще.

- **Требования к UX/UI реализации**

Интерфейс должен подстраиваться под любые устройства. Мало просто сделать так, чтобы блоки меняли размер — нужно продумать логику перестроения всего: от меню и таблиц до графиков и карточек компаний. Важно, чтобы на любом экране всё выглядело и работало предсказуемо.

Стандартные элементы UI-фреймворка лучше использовать через собственные обертки. Так мы приведем всё к единому стилю, сможем централизованно менять поведение компонентов и не превратим код в свалку. А если готовых решений не хватит — просто пишем свои элементы, строго придерживаясь нашей дизайн-системы и архитектуры.

- **Дополнительные требования к визуализации данных**

Так как платформа заточена под финансовую аналитику, интерфейс должен помогать быстро считывать цифры и графики. Пользователю важно наглядно видеть динамику доходов, структуру портфеля и любые другие показатели. При этом важно соблюдать единство стиля: графики должны одинаково хорошо смотреться и на десктопе, и на смартфоне, не теряя при этом своей информативности.

С таблицами тоже нужно поработать. Они должны легко переваривать большие массивы данных, позволяя быстро сортировать и фильтровать информацию. Чтобы глаз не

замыливался, стоит добавить акценты — например, подсвечивать рост или падение показателей цветом. Это поможет сразу заметить главное.

1.5. Нефункциональные требования

Я сформулировал нефункциональные требования к веб-платформе, чтобы система работала быстро, безопасно и была удобной для пользователей.

- **Скорость и производительность**

Мы ставим задачу: основные страницы должны открываться быстрее чем за 2 секунды, а любые манипуляции с данными (фильтры, сортировки, графики) — не дольше секунды. Чтобы этого добиться, нужно оптимизировать запросы к серверу и не перегружать загрузку данных.

- **Безопасность**

Защита данных и разграничение доступа — приоритет. Для входа используем современный стандарт OAuth 2.0, защищаем интерфейс от XSS-атак и на каждом закрытом маршруте обязательно проверяем права и роли пользователя.

- **Удобство (UX)**

Платформа должна быть интуитивной. Мы стремимся к тому, чтобы любое действие занимало не более 3–4 кликов. Интерфейс должен быть единообразным, а система — всегда давать обратную связь: уведомлять об успехе, ошибках или предупреждать о нюансах. В формах ввода обязательно добавим подсказки и валидацию, чтобы пользователь сразу видел, если что-то заполнил не так.

- **Масштабируемость и рост**

Система должна расти вместе с бизнесом без переписывания архитектуры с нуля. Мы закладываем возможность легко подключать новые модули. Чтобы большие объемы данных не «вешали» систему, внедрим пагинацию и ленивую загрузку.

- **Совместимость**

Продукт должен одинаково хорошо работать везде: от смартфонов и планшетов до десктопов, на любых разрешениях экрана. Поддерживаем актуальные версии Chrome, Firefox, Edge и Safari.

- **Поддержка и обслуживание**

Чтобы разработка и исправление багов не превратились в кошмар, мы сделаем упор на чистоту кода и понятную структуру проекта. Также обязательно настроим логирование ошибок на стороне интерфейса — это поможет быстро находить и устранять проблемы.

1.6. Функциональные требования:

Основные требования к нашей веб-платформе для анализа финансов публичных компаний.

- **Регистрация и личный кабинет**

Нам нужно, чтобы пользователи могли создавать аккаунты и безопасно хранить свои данные. При регистрации система должна проверять корректность почты и заполненность полей, а при входе — пускать пользователя в закрытые разделы. У каждого клиента должен быть личный кабинет с настройками и возможностью выйти из системы. Если пользователь ошибся при вводе пароля, нужно показать простое и понятное предупреждение, не вываливая на него лишние технические подробности.

- **Работа с данными компаний**

Главная задача здесь — дать удобный доступ к информации о компаниях. В системе будет общий список, где любую организацию можно быстро найти по названию или тикеру. Для каждой компании мы сделаем отдельную карточку: сначала общая справка, затем ключевые финансовые показатели. Информация должна быть структурирована так, чтобы можно было легко перейти от общего обзора к деталям. Если каких-то данных по компании не хватает, система должна просто вежливо об этом сообщить.

- **Анализ финансовых результатов**

Чтобы пользователи могли оценивать динамику бизнеса, платформа должна показывать финансовые метрики за несколько периодов. Мы добавим возможность отслеживать выручку, прибыль, долги и рентабельность. Для наглядности данные будут представлены и в таблицах, и в виде графиков — так тренды роста или падения считываются гораздо быстрее. Также система должна сама рассчитывать основные финансовые коэффициенты, помогая пользователю сразу видеть важные изменения в показателях.

- **Личный учет сделок**

Чтобы пользователи могли самостоятельно вести учет своих операций, мы добавили функционал управления сделками. При регистрации новой операции нужно указать тип сделки, актив, дату, цену и количество. Все эти данные сохраняются и автоматически идут в расчеты портфеля. Если вы ошиблись при вводе — не проблема: любую запись можно отредактировать или вовсе удалить. Вся история операций всегда под рукой в удобном и наглядном виде.

- **Управление портфелем**

Инструменты управления портфелем помогают контролировать активы и видеть реальный финансовый результат. Система сама строит структуру портфеля на основе ваших сделок. Для каждого актива мы считаем объем, среднюю цену покупки и текущую стоимость. Вы сможете оценить доходность как по отдельным позициям, так и по всему портфелю в целом. Благодаря интеграции с рыночными данными система в реальном времени показывает текущую прибыль или убыток и то, как меняется стоимость ваших активов во времени.

- **Визуализация данных**

Чтобы цифры не превращались в скучные таблицы, мы используем графики и диаграммы. Это помогает мгновенно считывать динамику показателей за разные периоды. В интерфейсе мы используем цветовое выделение: так сразу видно, где идет рост, а где просадка. Графики работают везде — и в аналитике компаний, и в личном портфеле, помогая быстрее принимать решения на основе визуальных данных.

- **Фильтрация и сортировка**

Работать с большими массивами данных проще, если использовать фильтры. Вы можете сортировать список компаний по любым значимым параметрам или искать конкретные сделки, отсеивая их по типу операции, активу или дате. Все изменения применяются мгновенно, а интерфейс поиска интуитивно понятен и не требует долгого обучения.

- **Уведомления и обратная связь**

Система всегда «на связи» и подтверждает каждое ваше действие: от успешного входа в аккаунт до сохранения новой сделки. Если что-то пойдет не так, вы получите понятное сообщение об ошибке без лишнего технического жаргона. Если данные введены некорректно, система подскажет, где именно нужно внести исправления. Все уведомления заметны, но не перегружают интерфейс, обеспечивая прозрачный и комфортный процесс работы.

Глава 2. Обзор существующих решений

Перед разработкой собственной веб-платформы для анализа финансовых результатов публичных компаний необходимо рассмотреть уже существующие цифровые решения, используемые частными инвесторами и аналитиками. Такой обзор позволяет определить сильные и слабые стороны аналогов, выявить наиболее востребованные функции, а также понять, какие подходы и интерфейсные решения целесообразно использовать в разрабатываемой системе.

Современный рынок финансовых сервисов предлагает большое количество платформ, ориентированных на отображение рыночных данных, фундаментальный анализ компаний, ведение инвестиционного портфеля и визуализацию ключевых показателей. Однако большинство таких решений либо ориентировано только на один аспект работы с данными, либо обладает избыточной сложностью, либо требует платной подписки для получения доступа к важным возможностям. В связи с этим актуальной задачей является создание веб-платформы, объединяющей основные функции анализа и учета пользовательских данных в едином интерфейсе.

2.1. TradingView

TradingView является одной из наиболее популярных платформ для анализа финансовых рынков. Основное внимание в данном сервисе уделяется визуализации рыночной

информации, работе с графиками и техническому анализу. Пользователю предоставляется доступ к широкому набору интерактивных графиков, инструментов рисования, индикаторов, а также к информации по акциям, валютам, индексам и другим активам.

Сильной стороной TradingView является удобный и современный интерфейс, а также высокая наглядность представления данных. Пользователь может быстро получить доступ к котировкам, динамике цен и ключевым рыночным метрикам. Платформа хорошо адаптирована как для профессиональных аналитиков, так и для частных инвесторов. Вместе с тем фундаментальный анализ финансовой отчетности в TradingView представлен ограниченно по сравнению со специализированными аналитическими сервисами. Кроме того, часть функций недоступна без оформления платной подписки.

Решения, которые можно взять:

1. удобная визуализация данных с помощью интерактивных графиков;
2. современный и лаконичный интерфейс;
3. структурированное отображение информации об активе;
4. использование фильтров и переключателей временных интервалов;
5. выделение ключевых показателей в отдельных информативных блоках.

2.2. Yahoo Finance

Yahoo Finance — это широко известный онлайн-сервис, предоставляющий финансовую информацию по публичным компаниям, биржевым инструментам и рыночным индексам. Платформа предлагает пользователям данные о стоимости акций, рыночной капитализации, отчетности, новостях и некоторых аналитических показателях. Одним из преимуществ сервиса является открытый доступ к большому объему информации и простота использования.

Платформа позволяет быстро находить компании по названию или тикеру, просматривать исторические данные и отслеживать базовые рыночные показатели. Также Yahoo Finance поддерживает создание пользовательских списков наблюдения. Однако аналитические возможности системы ограничены, а глубина проработки финансовой отчетности и пользовательских инструментов уступает специализированным решениям.

Решения, которые можно взять:

1. быстрый поиск компаний по тикеру и названию;
2. карточки компаний с краткой сводной информацией;
3. разделение данных на вкладки: обзор, финансовые показатели, новости, история;
4. удобная структура пользовательского наблюдения за активами;

5. простая и понятная навигация между разделами.

2.3. Investing.com

Investing.com представляет собой крупную финансовую платформу, сочетающую рыночные данные, новости, экономический календарь и базовые аналитические инструменты. Сервис ориентирован на широкий круг пользователей и предоставляет доступ к информации по акциям, валютам, сырьевым товарам, облигациям и индексам. Одной из сильных сторон платформы является большое количество доступных источников информации и высокая информативность страниц активов.

Пользователь может просматривать котировки, динамику, новости, аналитику и прогнозы, а также пользоваться календарем экономических событий. Вместе с тем интерфейс сервиса местами перегружен информацией, что может затруднять восприятие данных, особенно для начинающих пользователей. Кроме того, большое количество второстепенного контента может отвлекать от основной аналитической задачи.

Решения, которые можно взять:

1. объединение аналитики, новостей и рыночных данных в одной системе;
2. использование сводных блоков с ключевой информацией;
3. отображение связанных событий, которые могут влиять на финансовые показатели;
4. возможность быстрого перехода между различными категориями данных;
5. представление исторической информации в удобной табличной форме.

2.4. Simply Wall St

Simply Wall St — это аналитическая платформа, ориентированная на наглядное представление фундаментальных данных о компаниях. Отличительной особенностью сервиса является активное использование визуализации: финансовая информация, риски, оценка стоимости компании и перспективы роста отображаются в виде диаграмм, карточек и инфографики. Такой подход делает сложные финансовые данные более понятными для широкой аудитории.

Платформа хорошо подходит для первичного анализа компаний, так как помогает быстро выделять сильные и слабые стороны бизнеса. Однако часть расчетов и выводов сервиса представляется в упрощенном виде, что может быть недостаточно для пользователей, которым требуется детальный доступ к исходным данным. Кроме того, многие функции ограничены платным доступом.

Решения, которые можно взять:

1. акцент на визуальном представлении сложной финансовой информации;
2. использование цветовых маркеров для оценки состояния показателей;
3. представление аналитики в виде карточек, диаграмм и кратких выводов;
4. упрощение восприятия финансовых коэффициентов для пользователя;
5. выделение рисков и преимуществ компании в отдельные блоки.

2.5. Tinkoff Инвестиции / брокерские платформы

Брокерские платформы, такие как Тинькофф Инвестиции, Альфа-Инвестиции и другие, ориентированы не только на предоставление рыночной информации, но и на совершение реальных операций с активами. Для пользователя они удобны тем, что сочетают информацию по инструментам, портфель, историю сделок и отслеживание финансового результата. Как правило, такие сервисы обладают простым и дружелюбным интерфейсом, хорошо адаптированным под массового пользователя.

Вместе с тем аналитика фундаментальных показателей компании в таких системах обычно не является основной задачей и представлена в сокращенном виде. Основной фокус сделан на удобстве торговли, состоянии портфеля и текущем результате инвестора.

Решения, которые можно взять:

1. удобное отображение инвестиционного портфеля;
2. наглядный расчет прибыли и убытка по активам;
3. представление истории сделок в структурированном виде;
4. простые формы добавления и просмотра пользовательских данных;
5. адаптивный интерфейс для мобильных и настольных устройств.

2.6. Сравнительный анализ существующих решений

Рассмотренные платформы демонстрируют различные подходы к представлению финансовой информации и взаимодействию с пользователем. Одни решения делают акцент на техническом анализе и графиках, другие — на фундаментальной отчетности, третьи — на удобстве управления личным портфелем. Это показывает, что универсального решения, в полной мере объединяющего удобный анализ публичных компаний и ведение пользовательских сделок в одном интерфейсе, на практике немного.

На основании проведенного обзора можно сделать вывод, что для разрабатываемой платформы целесообразно объединить наиболее удачные элементы существующих систем, адаптировав их под цели проекта.

Решения, которые можно взять в собственную разработку:

1. из TradingView — интерактивные графики и удобную визуализацию динамики;
2. из Yahoo Finance — простой поиск компаний и логичную структуру карточки компании;
3. из Investing.com — совмещение данных, новостей и аналитики в одном интерфейсе;
4. из Simply Wall St — наглядное представление финансовых коэффициентов и рисков;
5. из брокерских приложений — учет сделок, отображение портфеля и расчет финансового результата.

2.7. Вывод по главе

Проведенный обзор показал, что существующие решения обладают рядом полезных функций и удачных интерфейсных подходов, которые могут быть использованы при проектировании собственной веб-платформы. Наиболее ценными являются механизмы быстрого поиска компаний, наглядная визуализация финансовых данных, отображение ключевых коэффициентов, удобная работа с историей сделок и представление инвестиционного портфеля.

При этом ни одна из рассмотренных платформ не предлагает в полностью сбалансированном виде сочетание фундаментального анализа компаний, удобного учета собственных сделок пользователя и простого интерфейса, ориентированного на учебные и прикладные аналитические задачи. Это подтверждает целесообразность разработки собственной системы, которая будет учитывать сильные стороны существующих решений и устранять их недостатки.

Глава 3. Выбор инструментальных средств и технологий

В настоящее время существует большое количество технологий и программных средств, применяемых при разработке веб-приложений. Каждое из них обладает собственными преимуществами, ограничениями и областями наиболее эффективного применения. Выбор инструментов для разработки должен основываться не только на их популярности, но и на соответствии целям проекта, удобстве сопровождения, скорости разработки и возможностях масштабирования.

В данном разделе обосновывается выбор технологий, использованных при разработке веб-платформы для анализа финансовых результатов публичных компаний и учета пользовательских инвестиционных сделок.

- **React, Vite и TypeScript**

Для реализации клиентской части приложения был выбран стек React + Vite + TypeScript.

Библиотека React позволяет создавать современные пользовательские интерфейсы на основе компонентного подхода. Использование компонентов упрощает повторное применение элементов интерфейса, облегчает сопровождение кода и делает структуру приложения более понятной. React широко применяется при создании одностраничных веб-приложений, что делает его подходящим выбором для платформы с большим количеством интерактивных элементов, таблиц, форм и аналитических блоков.

Инструмент Vite был выбран в качестве среды сборки и запуска приложения. Его основным преимуществом является высокая скорость старта проекта и быстрая пересборка при внесении изменений в код. Это особенно важно в процессе активной разработки интерфейса, когда требуется оперативно проверять результат изменений. По сравнению с более тяжеловесными сборщиками Vite обеспечивает более комфортную и производительную среду разработки.

Для повышения надежности кода был использован TypeScript. Данная технология расширяет возможности JavaScript за счет статической типизации, что позволяет обнаруживать многие ошибки еще на этапе разработки. Применение TypeScript особенно полезно в проектах, где присутствует большое количество объектов данных, ответов API, пользовательских сущностей и вычисляемых значений. Использование типизации делает код более предсказуемым, облегчает рефакторинг и улучшает читаемость проекта.

- **Express.js и Node.js**

Для реализации серверной части приложения были выбраны Node.js и Express.js с использованием TypeScript.

Среда выполнения Node.js позволяет разрабатывать серверную часть на языке JavaScript/TypeScript, что упрощает взаимодействие между фронтендом и бэкендом за счет использования единого технологического стека. Это делает проект более целостным и снижает порог сложности при сопровождении кода.

Фреймворк Express.js был выбран как легковесное и гибкое средство для создания REST-сервера. Он предоставляет базовые механизмы маршрутизации, обработки HTTP-запросов, подключения промежуточных обработчиков и организации серверной логики. Express.js хорошо подходит для построения прикладного API, обеспечивающего работу с пользователями, компаниями, портфелем и сделками.

Использование TypeScript на серверной стороне также повышает надежность кода и позволяет точнее описывать структуры запросов, ответов и моделей данных.

- **PostgreSQL**

В качестве системы управления базами данных была выбрана PostgreSQL.

PostgreSQL является одной из наиболее надежных и функциональных реляционных СУБД. Она хорошо подходит для проектов, в которых необходимо хранить структурированные данные, связанные между собой отношениями. В рамках данного проекта база данных используется для хранения информации о пользователях, сделках, параметрах портфеля и других сущностях системы.

Реляционная модель данных удобна для реализации запросов, связанных с выборкой, фильтрацией, сортировкой и агрегацией информации. Это особенно важно для аналитической платформы, где требуется работать с финансовыми данными и историей пользовательских операций.

Дополнительно преимуществом PostgreSQL является устойчивость, поддержка сложных запросов и хорошая совместимость с современными ORM-инструментами.

- **Drizzle ORM**

Для взаимодействия серверной части с базой данных была использована Drizzle ORM.

Drizzle ORM представляет собой современный инструмент для работы с реляционными базами данных в TypeScript-проектах. Одной из его основных особенностей является типобезопасный подход к описанию схемы базы данных и выполнению запросов. Это позволяет уменьшить количество ошибок, связанных с несоответствием структуры данных, и упрощает сопровождение проекта.

Использование Drizzle ORM делает код доступа к базе данных более прозрачным и управляемым. В отличие от более сложных ORM-решений, данный инструмент отличается относительной легковесностью и хорошей интеграцией с TypeScript.

- **Passport.js и express-session**

Для реализации аутентификации и управления пользовательскими сессиями были выбраны Passport.js и express-session.

Библиотека Passport.js применяется для организации авторизации пользователей. В проекте используется Local Strategy, то есть вход в систему по логину и паролю. Такой подход является понятным, широко распространенным и достаточным для задач разрабатываемой платформы. Пакет express-session обеспечивает хранение пользовательской сессии на сервере, благодаря чему после успешной авторизации пользователь может продолжать работу в системе без необходимости повторного ввода данных при каждом запросе. Такой механизм удобен для приложений, в которых важны персонализированные функции, например работа с личным кабинетом, сделками и инвестиционным портфелем.

- **TanStack Query v5**

Для работы с серверными запросами и кэшированием мы выбрали TanStack Query v5. Эта библиотека берет на себя всю рутину: получение, обновление и повторную загрузку данных. Для нашей платформы, где постоянно меняются котировки, история сделок и состав портфелей, это критически важно. С TanStack Query гораздо проще управлять состояниями загрузки и ошибками, а интерфейс работает заметно отзывчивее. В итоге код стал чище, а пользователь получает актуальную информацию без лишних задержек.

- ***shadcn/ui и Tailwind CSS***

Интерфейс я собрал на shadcn/ui и Tailwind CSS. С shadcn/ui всё просто: это готовые компоненты, которые уже выглядят современно и удобно. Они экономят кучу времени на разработку и помогают выдержать единый стиль во всем приложении. Tailwind CSS взял для стилизации — он позволяет быстро накидывать нужный вид прямо через классы, что очень удобно для создания адаптивного дизайна. И, конечно, добавил темную тему: сейчас это стандарт, да и при долгой работе с аналитикой глазам будет гораздо приятнее.

- **Wouter**

Для маршрутизации на клиентской стороне была выбрана библиотека Wouter.

Wouter является легковесным решением для организации навигации в React-приложениях. В отличие от более крупных маршрутизаторов, он предоставляет только необходимые базовые возможности, не перегружая проект избыточной функциональностью. Для данного проекта

такой подход является оправданным, поскольку приложение не требует чрезмерно сложной клиентской маршрутизации.

Использование Wouter позволяет организовать переходы между страницами платформы, например между главной страницей, карточкой компании, разделом сделок, портфелем и личным кабинетом.

- **Tinkoff Invest API**

В качестве внешнего источника финансовых данных был использован Tinkoff Invest API.

Данный API предоставляет доступ к информации о финансовых инструментах и рыночных данных по протоколу HTTPS REST. Использование внешнего API позволяет получать актуальные данные, необходимые для отображения информации о компаниях, котировках и связанных аналитических показателях.

Подключение Tinkoff Invest API делает платформу более практической, поскольку данные поступают из реального внешнего источника, а не задаются вручную. Это повышает прикладную ценность системы и приближает ее к реальным условиям эксплуатации.

- **REST API**

Взаимодействие между клиентской и серверной частями приложения построено на основе REST API.

REST-подход является одним из наиболее распространенных стандартов организации обмена данными между фронтендом и бэкендом. Он предполагает использование HTTP-запросов для выполнения операций получения, создания, изменения и удаления данных. Для данного проекта такой подход удобен, поскольку позволяет логично разделить клиентскую и серверную части системы.

REST API используется для работы с пользователями, сделками, портфелем, данными компаний и другими сущностями приложения.

- **Figma**

Для проектирования интерфейсов и предварительной подготовки визуальных решений использовалась Figma.

Данный инструмент позволяет разрабатывать макеты экранов, проектировать пользовательские сценарии и согласовывать внешний вид приложения до этапа программной реализации. Использование Figma способствует более структурированной разработке интерфейса и снижает количество доработок на поздних этапах проекта.

- **Git и GitHub**

Для контроля версий и хранения исходного кода использовались Git и GitHub.

Система Git позволяет фиксировать историю изменений, работать с ветками разработки и при необходимости возвращаться к предыдущим версиям проекта. Размещение репозитория на GitHub обеспечивает централизованное хранение кода и удобство резервного копирования. Применение этих инструментов особенно важно в процессе поэтапной разработки, тестирования и сопровождения приложения.

Глава 4. Реализация

4.1 Разработка дизайн-макетов интерфейса в Figma

Перед началом программной реализации были разработаны дизайн-макеты пользовательского интерфейса в среде Figma. Создание макетов позволило заранее определить структуру экранов, логику пользовательских сценариев, визуальный стиль приложения и единые подходы к оформлению элементов

интерфейса. Это упростило дальнейшую верстку и обеспечило целостность пользовательского опыта.

Общий визуальный стиль проекта выполнен в темной цветовой гамме с акцентом на фиолетовые и сиреневые оттенки, что соответствует современной стилистике финансовых и аналитических платформ. Основной фон интерфейса имеет почти черный цвет с мягкими градиентными переходами, благодаря чему внимание пользователя концентрируется на ключевых функциональных элементах. В качестве акцентного цвета используется яркий фиолетовый, применяемый для кнопок, логотипа, активных элементов и отдельных декоративных деталей.

• **Экран авторизации**

Макет экрана авторизации выполнен в минималистичном стиле. В центральной части страницы расположена компактная форма входа, оформленная в виде затемненной полупрозрачной карточки со скругленными углами. В верхней части карточки размещен логотип платформы Ledgerly, выполненный в фиолетовой цветовой палитре. Ниже располагается поясняющий текст, информирующий пользователя о назначении формы.

Форма содержит стандартные поля для ввода электронной почты и пароля, оформленные в светлом цвете на темном фоне, что обеспечивает хороший визуальный контраст и удобство восприятия. Кнопка входа выделена насыщенным фиолетовым цветом и является главным акцентным элементом формы. В нижней части экрана предусмотрена ссылка для перехода к регистрации.

Особенностью данного макета является декоративный фон: правая часть экрана содержит крупную иллюстрацию в виде стилизованных горных форм, выполненных в серо-фиолетовых оттенках. Такой графический элемент делает интерфейс более выразительным и визуально запоминающимся, не перегружая его лишними деталями.

• **Экран восстановления пароля**

Макет страницы восстановления пароля выполнен в том же визуальном стиле, что и экран авторизации, благодаря чему сохраняется единая концепция

интерфейса. В центре страницы располагается карточка с логотипом, заголовком Reset Password и кратким пояснением для пользователя.

В форме предусмотрено одно поле для ввода адреса электронной почты, на который должна быть отправлена инструкция по восстановлению доступа. Основная кнопка действия также оформлена в фирменном фиолетовом цвете. В верхней части карточки размещена ссылка для возврата к экрану входа. Повторное использование тех же цветов, отступов, форм полей и кнопок обеспечивает целостность интерфейса и упрощает восприятие пользователем сценариев аутентификации.

- **Экран регистрации**

Страница регистрации визуально повторяет общую структуру предыдущих экранов, однако содержит более сложную форму. В ней предусмотрены поля для ввода имени, фамилии, адреса электронной почты и пароля, а также дополнительные переключатели, связанные с пользовательскими настройками.

Форма регистрации размещена в центральной части экрана в карточке с затемненным фоном и скругленными границами. Несмотря на большее количество элементов, макет сохраняет чистоту композиции и логичную последовательность ввода данных. Основная кнопка завершения регистрации оформлена в том же акцентном стиле, что и на других страницах. Нижняя часть карточки содержит ссылку для перехода к авторизации, если учетная запись уже существует.

Единое оформление всех страниц аутентификации позволяет сформировать у пользователя целостное впечатление о системе и облегчает освоение интерфейса на начальном этапе работы с платформой.

- **Экран поиска и выбора компании**

После входа в систему пользователь попадает на экран, связанный с выбором и поиском публичных компаний. Данный макет уже отражает основную рабочую часть платформы. Интерфейс разделен на две основные области: боковую навигационную панель и центральную рабочую область.

Слева располагается вертикальное меню с логотипом платформы, строкой поиска и списком доступных компаний. Каждая компания представлена в виде отдельного элемента списка с круглой иконкой и тикером. Активный элемент выделяется фиолетовым цветом, что позволяет пользователю быстро определить выбранную сущность. Такая боковая структура делает навигацию по списку компаний удобной и понятной.

Центральная часть экрана предназначена для отображения информации по выбранной компании. В представленном макете в центре страницы отображается круглый логотип компании, ее название и тикер. Ниже размещено поле поиска или выбора, позволяющее уточнить запрос. Такое композиционное решение делает основной акцент на выбранной компании и подготавливает пользователя к переходу к более детальному аналитическому представлению данных.

- **Экран инвестиционного портфеля**

Одним из ключевых макетов системы является экран инвестиционного портфеля. Он предназначен для отображения информации о пользовательских активах, финансовом результате и совершенных сделках.

Как и в предыдущем экране, слева расположена боковая панель со списком компаний и навигационными элементами. Основная рабочая область организована более сложно и содержит несколько информационных блоков. В верхней части экрана размещены сводные показатели, отражающие текущее состояние инвестиционного портфеля, например абсолютное изменение стоимости и относительную доходность. Эти данные оформлены в виде отдельных карточек, что повышает удобство восприятия информации.

Справа в верхней части расположен элемент добавления новой сделки, оформленный как акцентная кнопка. Ниже находятся средства фильтрации и сортировки, позволяющие управлять отображением сделок по различным параметрам. Основная часть страницы отведена под табличное представление операций. В таблице отображаются все сделки пользователя. Если истории операций пока нет, система выведет подсказку.

Интерфейс портфеля получился строгим и лаконичным — именно таким, каким должен быть аналитический инструмент. Темная тема в сочетании с цветовой индикацией прибыли и убытков помогает мгновенно считывать показатели, не перегружая зрение. Работа над макетами в Figma помогла нам заранее продумать всё: от логики переходов до визуального стиля. Это не только задало единый тон всему приложению, но и значительно упростило процесс разработки, так как у программистов был четкий и понятный план.

4.2 Структура проекта

```
✓ LEDGERLY
  > .agents
  > .config
  > .local
  > attached_assets
  > client
  > dist
  > migrations
  > node_modules
  > script
  > scripts
  > server
  > shared
  ⚙ .env
  ⚡ .gitignore
```

```
{ } components.json
TS drizzle.config.ts
{ } package-lock.json
{ } package.json
JS postcss.config.js
TS tailwind.config.ts
≡ term.txt
TS tsconfig.json
⚡ vite.config.ts
```

Одной из важных задач реализации веб-приложения стала организация удобной структуры проекта. Архитектура исходного кода была построена таким образом, чтобы разделить клиентскую и серверную части, выделить общие модули, упростить сопровождение приложения и обеспечить возможность дальнейшего расширения функциональности.

На верхнем уровне проект включает набор директорий и конфигурационных файлов, отвечающих за различные аспекты работы системы: интерфейс пользователя, серверную бизнес-логику, взаимодействие с базой данных, миграции, скрипты автоматизации и параметры сборки.

- **Общая логика построения проекта**

Проект организован по модульному принципу и условно разделен на несколько крупных частей:

1. клиентская часть;
2. серверная часть;
3. общие модули;
4. данные и миграции базы данных;
5. служебные и конфигурационные файлы.

Такое разделение позволяет изолировать зоны ответственности между различными частями системы. Клиентская часть отвечает за отображение интерфейса и взаимодействие с пользователем, серверная — за обработку запросов, работу с базой данных и бизнес-логику, а общие модули используются повторно в нескольких слоях приложения.

- **Каталог client**

Директория client содержит исходный код клиентской части приложения. Именно здесь располагаются компоненты интерфейса, страницы, маршрутизация, логика работы с пользовательскими действиями, а также механизмы взаимодействия с серверным API.

Клиентская часть реализована на основе React с использованием TypeScript, что позволяет создавать интерактивный и типобезопасный пользовательский интерфейс. Внутри данного каталога, как правило, размещаются компоненты, формы, таблицы, карточки отображения компаний, экраны авторизации, страницы инвестиционного портфеля и иные элементы визуального слоя системы.

- **Каталог server**

Директория server предназначена для хранения серверной логики приложения. В ней располагаются обработчики запросов, маршруты API, модули аутентификации, логика доступа к данным, а также функциональность, связанная с обработкой бизнес-сценариев.

Серверная часть реализована на базе Node.js и Express.js. Именно этот каталог обеспечивает связь между пользовательским интерфейсом, базой данных и внешними сервисами. В рамках проекта сервер отвечает за регистрацию и авторизацию пользователей, управление компаниями, обработку инвестиционных сделок, расчет отдельных показателей и выдачу информации клиентской части в структурированном формате.

- **Каталог shared**

В директории `shared` мы храним всё то, что нужно и клиенту, и серверу одновременно. Это могут быть типы данных, интерфейсы, схемы валидации или модели — в общем, любые элементы, которые должны быть общими для всего приложения. Такой подход избавляет от дублирования кода и помогает фронтенду и бэкенду «говорить на одном языке». Если структура пользователя или сделки описана в одном месте, риск ошибиться при передаче данных между слоями приложения сводится к минимуму.

- **Каталог `migrations`**

В папке `migrations` лежат миграции — это пошаговые инструкции по созданию и изменению структуры базы данных. Такой подход критически важен при работе с реляционными БД: вы всегда видите историю изменений схемы и можете легко развернуть идентичную структуру базы на любом другом сервере. Если вы видите отдельный каталог для миграций, значит, проект проектировался всерьез — с прицелом на нормальное развертывание и поддержку, а не просто как быстрый прототип на коленке.

- **Каталог `dist`**

В папке `dist` лежат готовые результаты сборки. Здесь хранятся скомпилированные и оптимизированные файлы, которые можно сразу запускать или деплоить. Обычно содержимое этой директории создается автоматически, поэтому руками туда лучше не лезть. Если в проекте есть такой каталог, значит, процесс подготовки приложения к работе вынесен в отдельный этап.

- **Каталог `node_modules`**

Директория `node_modules` содержит установленные зависимости проекта. Это служебный каталог, в который менеджер пакетов помещает сторонние библиотеки и модули, необходимые для работы приложения. Хотя данная папка является обязательной для запуска проекта, она не содержит непосредственно бизнес-логику разрабатываемой системы.

- **Каталоги `scripts` и `script`**

Директории `scripts` и `script` используются для вспомогательных программных сценариев. В них могут размещаться утилиты автоматизации, скрипты подготовки данных, вспомогательные команды для разработки, импорта информации или обслуживания проекта.

Выделение таких скриптов в отдельные папки делает проект более организованным и позволяет не смешивать служебную логику с основным кодом клиентской и серверной частей.

- **Каталог `attached_assets`**

Директория `attached_assets` предназначена для хранения подключаемых ресурсов проекта. В подобных каталогах обычно размещаются изображения, иконки, графические материалы, вспомогательные файлы интерфейса или иные статические ресурсы, используемые в приложении.

- **Служебные каталоги `.config`, `.local`, `.agents`**

На верхнем уровне проекта также присутствуют служебные директории `.config`, `.local` и `.agents`. Они используются для локальных настроек окружения, внутренней конфигурации среды разработки или вспомогательных инструментов. Подобные каталоги не относятся напрямую к бизнес-функциям приложения, однако поддерживают корректную работу среды, автоматизации и дополнительных механизмов разработки.

- **Конфигурационные файлы проекта**

Помимо каталогов, на верхнем уровне проекта размещен ряд файлов конфигурации, определяющих параметры сборки, стилизации, подключения зависимостей и взаимодействия с базой данных.

1. Файл `package.json` является одним из ключевых файлов Node.js-проекта. Он содержит информацию о зависимостях, доступных командах запуска, сборки и разработки, а также общие сведения о приложении. Файл `package-lock.json` фиксирует точные версии установленных пакетов, обеспечивая воспроизводимость окружения.
2. Файл `tsconfig.json` задает параметры компиляции TypeScript и определяет правила обработки исходного кода.
3. Файл `vite.config.ts` содержит настройки сборщика Vite, который используется для разработки и сборки клиентской части приложения.
4. Файлы `tailwind.config.ts` и `postcss.config.js` отвечают за настройку системы стилизации и обработки CSS, в частности за работу Tailwind CSS.
5. Файл `drizzle.config.ts` содержит конфигурацию для Drizzle ORM и связан с организацией взаимодействия приложения с базой данных, а также с выполнением миграций.
6. Файл `components.json` используется для настройки библиотеки пользовательских интерфейсных компонентов и определяет параметры генерации или подключения визуальных элементов.
7. Файл `.env` содержит переменные окружения, например строки подключения к базе данных, параметры сессий, ключи API и другие чувствительные настройки. Вынос

таких данных в отдельный файл позволяет не хранить конфиденциальную информацию непосредственно в исходном коде.

8. Файл `.gitignore` определяет, какие файлы и каталоги не должны попадать в систему контроля версий Git. Это необходимо для исключения временных, служебных и конфиденциальных данных из репозитория.

- **Принципы организации структуры**

Таким образом, структура проекта построена по модульному принципу и ориентирована на разработку полноценного клиент-серверного веб-приложения. Такое построение обеспечивает удобство дальнейшей разработки, упрощает сопровождение кода и создает основу для расширения функциональных возможностей системы.

4.3 База данных

Файл `shared/schema.ts` является единым источником правды для типов данных: он используется как на сервере (Drizzle ORM), так и на клиенте (TypeScript-типы через Zod).

- **Таблица пользователей**

```
export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  email: text("email").notNull().unique(),
  password: text("password").notNull(),
  firstName: text("first_name").notNull(),
  lastName: text("last_name").notNull(),
  phoneNumber: text("phone_number").notNull(),
});
```

- **Таблица компаний (companies)**

Аналитический справочник — статичные данные о компаниях, загруженные вручную.

```
export const companies = pgTable("companies", {
  id: serial("id").primaryKey(),
  ticker: text("ticker").notNull().unique(),
  name: text("name").notNull(),
  description: text("description").notNull(),
  sector: text("sector").notNull(),
  logoColor: text("logo_color").notNull(),
  logoUrl: text("logo_url"),
});
```

- **Таблица мультипликаторов (key_ratios)**

Связь один к одному с companies. Хранит 12 финансовых показателей.

```
export const keyRatios = pgTable("key_ratios", {
  id: serial("id").primaryKey(),
  companyId: integer("company_id").references(() => companies.id).notNull(),
  payoutRatio: text("payout_ratio"),
  dividendYield: text("dividend_yield"),
  avgDivYield: text("avg_div_yield"),
  roa: text("roa"),
  roe: text("roe"),
  netDebtEbitda: text("net_debt_ebitda"),
  netDebtCapital: text("net_debt_capital"),
  evEbitda: text("ev_ebitda"),
  ps: text("ps"),
  pe: text("pe"),
  pbv: text("pbv"),
  revenueGrowth: text("revenue_growth"),
  ebitdaGrowth: text("ebitda_growth"),
  netProfitGrowth: text("net_profit_growth"),
});
```

- **Таблица прогнозов аналитиков (analyst_forecasts)**

Связь один ко многим с companies.

```
export const analystForecasts = pgTable("analyst_forecasts", {
  id: serial("id").primaryKey(),
  companyId: integer("company_id").references(() => companies.id).notNull(),
  analystName: text("analyst_name").notNull(),
  recommendation: text("recommendation").notNull(), // BUY, HOLD, SELL
  targetPrice: text("target_price").notNull(),
});
```

- **Таблица сделок (trades)**

Журнал сделок — связь один ко многим с users. Каждый пользователь видит только свои сделки.

```
export const trades = pgTable("trades", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").references(() => users.id).notNull(),
  date: text("date").notNull(),
  company: text("company").notNull(),
  ticker: text("ticker").notNull(),
  type: text("type").notNull(),
  buyPrice: numeric("buy_price").notNull(),
  sellPrice: numeric("sell_price"),
  quantity: integer("quantity").notNull(),
  currentPrice: numeric("current_price"),
  comment: text("comment"),
  status: text("status").notNull().default("open"),
  sector: text("sector"),
  takeProfit: numeric("take_profit"),
  stopLoss: numeric("stop_loss"),
});
```

4.4 Авторизация

Используется Passport.js с LocalStrategy (email + пароль). Сессии хранятся в PostgreSQL через connect-pg-simple, что позволяет им переживать перезапуск Сервера.

- Хэширование паролей(shared/schema.ts)

Пароли хэшируются через алгоритм `scrypt` с 16-байтовой случайной солью.

Хранится в виде строки `"hash.salt"`.

```
export async function hashPassword(password: string) {
  const salt = randomBytes(16).toString("hex");
  const buf = (await scryptAsync(password, salt, 64)) as Buffer;
  return `${buf.toString("hex")}.${salt}`;
}
```

При входе используется `timingSafeEqual` для защиты от `timing`-атак:

```
async function comparePasswords(supplied: string, stored: string) {
  const [hashed, salt] = stored.split(".");
  const hashedBuf = Buffer.from(hashed, "hex");
  const suppliedBuf = (await scryptAsync(supplied, salt, 64)) as Buffer;
  return timingSafeEqual(hashedBuf, suppliedBuf);
}
```

- **Цикл аутентификации**

Запрос	Действие
POST /api/auth/register	Хэшируем пароль → сохраняем → req.login()
POST /api/auth/login	Passport проверяет → req.login() → возвращаем user
GET /api/auth/user	req.isAuthenticated() → текущий пользователь
POST /api/auth/logout	req.logout() → чистим сессию

4.5 Tinkoff Invest API — server/tinkoff.ts

Весь обмен с Tinkoff идёт напрямую через Node.js модуль `https` (без сторонних SDK).

Эндпоинт: `invest-public-api.tinkoff.ru:443`.

- **Очистка токена**

Токен берётся из переменной окружения `TINKOFF_TOKEN` и очищается от non-ASCII символов — частая проблема при копировании из браузера:

```
const token = raw.replace(/[\x20-\x7E]/g, "").trim();
```

- **Кэш списка акций (1 час)**

Список акций TQBR запрашивается один раз в час. Фильтр: `classCode === TQBR`, `apiTradeAvailableFlag === true`, `ticker !== HHRU`.

```
let sharesCache: { data: RussianShare[]; ts: number } | null = null;
const CACHE_TTL_MS = 60 * 60 * 1000;
```

```
export async function getAllRussianShares(): Promise<RussianShare[]> {
  if (sharesCache && Date.now() - sharesCache.ts < CACHE_TTL_MS) {
    return sharesCache.data;
  }
}
```

Tinkoff возвращает цену в формате { units: "123", nano: 500000000 } (целая и дробная части отдельно):

```
function quotationToNumber(units: string | number, nano: number): number {
  return Number(units) + nano / 1_000_000_000;
}
```

Promise.allSettled позволяет получить цены нескольких тикеров одновременно — ошибка одного не прерывает остальные:

```
const result = await tinkoffPost(
  "/tinkoff.public.invest.api.contract.v1.InstrumentsService/Shares",
  { instrumentStatus: "INSTRUMENT_STATUS_ALL" }
);
```

4.6 API-маршруты — server/routes.ts

Все маршруты проверяют аутентификацию через req.isAuthenticated() перед любым действием. Изоляция данных: каждый пользователь видит только свои сделки.

Метод	Путь	Описание
POST	/api/auth/register	Регистрация нового пользователя
POST	/api/auth/login	Вход
POST	/api/auth/logout	Выход
GET	/api/auth/user	Текущий авторизованный пользователь
GET	/api/companies	Все компании из БД
GET	/api/companies/:ticker	Компания + мультипликаторы + прогнозы
GET	/api/trades	Сделки текущего пользователя
POST	/api/trades	Создать сделку
PATCH	/api/trades/:id	Обновить сделку
DELETE	/api/trades/:id	Удалить сделку

GET	/api/tinkoff/shares	Все акции TQBR (кэш 1ч)
GET	/api/prices?tickers=...	Живые цены по тикерам

4.7 Фронтенд — App.tsx

Используется библиотека wouter. Все защищённые маршруты проверяют авторизацию — неавторизованный пользователь автоматически перенаправляется на /login.

/login	Login	Публичный (редиректит на /dashboard если авторизован)
/register	Register	Публичный
/forgot-password	ForgotPassword	Публичный
/dashboard	Dashboard	Защищённый
/comparison	Comparison	Защищённый
/calculator	CalculatorPage (Журнал сделок)	Защищённый

4.8 Дашборд — dashboard.tsx

Двухколоночный layout: левая панель (w-80) — список всех акций TQBR с поиском, правая — детали выбранной компании с мультипликаторами и прогнозами аналитиков.

Цены обновляются каждые 3 секунды через setInterval. При смене тикера таймер пересоздаётся.

Компонент CompanyLogo пробует три источника по приоритету:

Уровень 1 — локальный файл из logoMap (батчи логотипов, загруженные вами)

Уровень 2 — CDN Tinkoff: <https://invest-static.cdn-tinkoff.ru/logos/invest/{base}x160.png>

Уровень 3 — цветная заглушка с первыми двумя буквами тикера

4.9 Журнал сделок — calculator.tsx

Самый сложный компонент приложения. Включает: таблицу сделок с сортировкой и фильтрами, расчёт P&L в реальном времени, боковую панель акций, форму добавления/редактирования сделки.

- **Независимый скролл боковой панели**

Ключевые классы для изоляции скролла — `h-screen + overflow-hidden` на корневом контейнере:

```
<div className="h-screen flex flex-col overflow-hidden">
  <div className="flex flex-1 overflow-hidden min-h-0">
    <aside className="flex flex-col overflow-hidden">
      <div className="flex-1 overflow-y-auto"> {/* скролл акций */}
```

- **Логотипы в таблице сделок**

Для каждой строки ищем тикер в двух источниках, чтобы получить `logoName` для CDN-фолбэка:

```
const companyRecord = companies.find(c => c.ticker === t.ticker);
const tShare = tinkoffShares.find(s => s.ticker === t.ticker);

<CompanyLogo
  ticker={t.ticker}
  color={companyRecord?.logoColor || tShare?.logoBaseColor}
  logoName={tShare?.logoName} // ← CDN-фолбэк
/>
```

- **Расчёт P&L**

Для открытых сделок P&L считается по живой цене, для закрытых — по цене продажи:

```
function calcPnl(trade, livePrice) {
  const entry = parseFloat(trade.buyPrice);
  const exit = trade.status === "closed"
    ? parseFloat(trade.sellPrice)
    : (livePrice ?? entry);
  return (exit - entry) * trade.quantity;
}
```

- **Расчёт доходности (%)**

```
• function calcRet(trade, livePrice) {
•   const entry = parseFloat(trade.buyPrice);
•   const exit = trade.status === "closed"
•     ? parseFloat(trade.sellPrice) : (livePrice ?? entry);
•   return entry > 0 ? (exit - entry) / entry * 100 : 0;
• }
```

4.10 Страница сравнения — `comparison.tsx`

Статическая страница с таблицей 4 нефтяных компаний (Лукойл, Роснефть, Газпром нефть, Сургутнефтегаз). Ранжирует мультипликаторы и подсвечивает цветом лучшие значения.

```
function getRanks(values, lowerIsBetter) {
  const sorted = [...values].sort((a, b) =>
    lowerIsBetter ? a - b : b - a);
  return values.map(v => sorted.indexOf(v));
  // 0 = лучший, 3 = худший
}
```

Ранг	Цвет	Смысл
0 (лучший)	Зелёный (emerald-400)	Самое привлекательное значение
1 (второй)	Оранжевый (orange-400)	Хорошее значение
2–3	Серый (white/80)	Нейтральное / худшее

4.11 Хранилище логотипов (logoMap)

Логотипы добавляются батчами. Каждый батч регистрируется в обоих файлах: `dashboard.tsx` и `calculator.tsx`. На данный момент добавлено ~130 тикеров (батчи 1–10: ABIO → SMLT).

```
// 1. Импорт (в начале файла, ДО logoMap)
import smltLogo from "@assets/SMLT_1773509806640.png";

// 2. Запись в карту
const logoMap: Record<string, string> = {
  // ...
  SMLT: smltLogo,
};
```

Важное: тикеры с дефисом (например, TCSG-RM) требуют кавычек в ключе: "TCSG-RM": `tcsLogo`.

Глава 5. Методика и результаты тестирования

• Цели тестирования

Тестирование стало финальной стадией разработки: нам нужно было убедиться, что платформа не просто работает, а делает это стабильно и удобно. Мы проверяли всё — от базовой логики до того, насколько легко пользователю будет ориентироваться в интерфейсе.

Главный фокус был на ключевых сценариях. Мы прогоняли авторизацию, просмотр данных о компаниях, учет сделок и отображение портфеля, а также проверяли, как система подтягивает данные из внешних источников. Поскольку это прикладное веб-приложение, мы сделали ставку на ручное тестирование. Это позволило не просто найти ошибки в коде, но и оценить «человеческий» фактор: насколько логичны переходы между экранами, легко ли читаются цифры и не вызывает ли интерфейс раздражения при работе.

• Методика тестирования

В рамках данной работы применялась преимущественно ручная методика тестирования. Такой подход был выбран в связи с тем, что разрабатываемая система включает большое количество пользовательских сценариев, связанных с визуальным интерфейсом, формами ввода, отображением таблиц, навигацией между страницами и обработкой данных, получаемых от сервера и внешних API.

Ручное тестирование проводилось последовательно, по основным функциональным модулям системы. Для каждого сценария определялись:

1. исходные условия;
2. действия пользователя;
3. ожидаемый результат;
4. фактический результат;
5. наличие или отсутствие ошибок.

Проверка выполнялась в процессе локального запуска приложения и при последовательном прохождении ключевых сценариев работы пользователя. В ходе тестирования оценивались как позитивные сценарии, при которых пользователь вводит корректные данные, так и негативные сценарии, включающие некорректный ввод, отсутствие обязательных данных, переходы в недопустимые состояния и иные типовые ошибки.

Основной упор был сделан на следующие виды ручного тестирования:

1. функциональное тестирование — проверка корректности реализации пользовательских функций;
2. интерфейсное тестирование — проверка отображения страниц, форм, кнопок, таблиц и навигации;
3. тестирование валидации — проверка обработки некорректных и неполных пользовательских данных;
4. интеграционное тестирование — проверка взаимодействия клиентской части, серверной логики, базы данных и внешнего API;
5. регрессионное тестирование — повторная проверка уже реализованных функций после внесения изменений в код.

• Проверяемые функциональные модули

В процессе тестирования были выделены основные функциональные модули системы, имеющие наибольшее значение для конечного пользователя:

1. модуль регистрации и авторизации пользователя;
2. модуль поиска и выбора публичных компаний;
3. модуль просмотра информации о компании;
4. модуль учета инвестиционных сделок;
5. модуль отображения инвестиционного портфеля;
6. модуль взаимодействия с внешними финансовыми данными;
7. модуль навигации и пользовательского интерфейса.

Проверка указанных модулей позволила оценить систему как с точки зрения отдельных функций, так и с точки зрения целостного пользовательского сценария.

• Тестирование регистрации и авторизации

На первом этапе было проверено функционирование механизма регистрации и входа в систему. Проверка включала создание новой учетной записи, вход с корректными учетными данными, обработку неверного логина или пароля, а также проверку поведения формы при пустых полях.

В результате тестирования было установлено, что:

1. пользователь может успешно зарегистрироваться в системе при вводе корректных данных;
2. после успешной регистрации становится доступен вход в систему;
3. при вводе некорректных учетных данных система не предоставляет доступ;
4. формы корректно реагируют на отсутствие обязательных полей;
5. переходы между страницами входа, регистрации и восстановления пароля работают корректно.

Таким образом, модуль аутентификации продемонстрировал корректную работу в основных пользовательских сценариях.

• Тестирование поиска и выбора компании

Следующим этапом была проверка сценария поиска публичных компаний и выбора интересующего эмитента. В ходе тестирования оценивались:

1. отображение списка компаний;
2. корректность работы строки поиска;
3. переход к выбранной компании;
4. обновление основной области интерфейса после выбора.

Тесты прошли успешно: список компаний выводится без ошибок, а поиск отлично помогает ориентироваться в данных. Когда выбираешь конкретную компанию, интерфейс обновляется мгновенно — можно сразу переходить к анализу.

• Тестирование отображения информации о компании

Мы также проверили экран с данными о публичных компаниях. Смотрели на всё сразу: насколько полно подгружается информация, не «едет» ли верстка и насколько логично всё расположено. В целом, интерфейс работает отлично: данные отображаются корректно, а структура страницы интуитивно понятна. Критичных ошибок, которые мешали бы работе с модулем, мы не нашли.

• Тестирование учета инвестиционных сделок

Мы протестировали модуль учета инвестиционных сделок. Проверили всё: от создания новой записи до того, как данные отображаются в списке после сохранения.

В ходе ручной проверки выяснили, что всё работает как надо. Пользователь может спокойно добавлять сделки через интерфейс, данные сохраняются и подтягиваются в список без ошибок. Если сделок еще нет, система корректно показывает пустой экран, а при перезагрузке страницы всё внесенное остается на месте.

Отдельно прошлись по негативным сценариям. Например, если попытаться отправить пустую форму, приложение не даст сохранить некорректные данные и вовремя выдаст ошибку.

• Тестирование интеграции с внешним API

Поскольку приложение берет финансовые данные из внешнего источника, мы уделили особое внимание проверке API. Нам нужно было убедиться, что данные приходят вовремя, обрабатываются без ошибок, а интерфейс не «зависает», если загрузка затягивается. Тесты прошли успешно: приложение стабильно подтягивает информацию, а пользователь спокойно может продолжать работу, пока данные обновляются. Интеграция работает как надо.

• Тестирование интерфейса и адаптивности

Дополнительно была проведена ручная проверка пользовательского интерфейса с точки зрения удобства взаимодействия, визуальной согласованности и адаптации под различные размеры экрана. Проверялись:

1. корректность отображения основных страниц;
2. читаемость текста и данных;
3. доступность кнопок и элементов управления;
4. сохранение структуры интерфейса при изменении размеров окна браузера;
5. единообразие визуального оформления.

В результате было установлено, что интерфейс приложения сохраняет целостность, элементы расположены последовательно, а навигация между разделами остается понятной для пользователя.

• **Результаты тестирования**

По итогам проведенного ручного тестирования было установлено, что разработанная система в целом корректно реализует предусмотренные функциональные возможности и обеспечивает выполнение основных пользовательских сценариев.

Результаты тестирования показали следующее:

механизмы регистрации, авторизации и переходов между связанными страницами работают корректно;

интерфейсы форм обрабатывают действия пользователя в ожидаемом порядке;

просмотр списка компаний и переход к выбранной компании выполняются без критических ошибок;

данные о компаниях, сделках и портфеле отображаются корректно;

система сохраняет и отображает пользовательские инвестиционные сделки;

интеграция с внешними финансовыми данными работает стабильно в рамках протестированных сценариев;

интерфейс приложения является логичным, целостным и удобным для использования;

критических ошибок, делающих невозможным использование системы по назначению, выявлено не было.

Выявленные в процессе разработки и промежуточных проверок недочеты носили локальный характер и были устранены до завершения итогового тестирования.

Глава 6. Пути возможного развития

У системы большой потенциал для роста, и мы видим несколько путей ее развития.

Во-первых, можно углубить аналитику. Сейчас проект фокусируется на базовых вещах, но в будущем в него легко добавить мультипликаторы, показатели рентабельности, ликвидности и долговой нагрузки. Это даст инвестору гораздо более объемную картину состояния компаний.

Во-вторых, стоит превратить систему в полноценный инструмент для управления портфелем. Было бы здорово добавить расчет средней цены покупки, отслеживание текущей доходности (P&L) и наглядную визуализацию активов.

Чтобы данные были полнее, мы планируем подключать новые API, новостные агрегаторы и прямые источники отчетности. Параллельно с этим нужно доработать интерфейс: сделать его более адаптивным, добавить графики, фильтры и систему уведомлений, чтобы работать с платформой было просто и приятно.

Техническая часть тоже требует внимания. Для реального использования необходимо усилить безопасность — внедрить надежную аутентификацию, защиту сессий и настроить регулярное резервное копирование. Поскольку сейчас мы тестируем всё вручную, следующим шагом станет переход к автоматизации: модульным, интеграционным и end-to-end тестам. Это поможет выпускать обновления быстрее и без багов.

Наконец, нужно заложить фундамент для масштабирования. Оптимизация базы данных, кэширование и контейнеризация позволят системе стабильно работать, даже когда количество пользователей вырастет в разы.

Заключение

В рамках выпускной квалификационной работы было создано веб-приложение Ledgerly, которое предоставляет пользователям удобный инструментарий для аналитики финансовых результатов публичных компаний и учета собственных сделок на бирже.

Разработанная веб-платформа построена на современном и масштабируемом стеке технологий.

- React, Vite и TypeScript обеспечивают создание быстрого, интерактивного и расширяемого клиентского интерфейса. Node.js и Express.js позволяют реализовать гибкую и удобную серверную архитектуру для обработки пользовательских запросов и организации REST API.
- PostgreSQL обеспечивает надежное хранение структурированных данных, связанных с пользователями, компаниями, сделками и инвестиционным портфелем.
- Drizzle ORM упрощает взаимодействие с базой данных и повышает типобезопасность серверной логики.
- TanStack Query позволяет эффективно работать с асинхронными запросами, кэшированием и обновлением данных на клиентской стороне.

- shadcn/ui и Tailwind CSS ускоряют разработку современного, адаптивного и визуально целостного пользовательского интерфейса.
 - Tinkoff Invest API предоставляет доступ к актуальным данным о финансовых инструментах и повышает практическую значимость системы.
1. Главными преимуществами разработанной платформы являются доступность, удобство и функциональность:
 2. анализ финансовых результатов публичных компаний;
 3. хранение и учет пользовательских инвестиционных сделок;
 4. отображение инвестиционного портфеля;
 5. современный и адаптивный интерфейс;
 6. наглядное представление данных;
 7. расширяемая архитектура приложения.

В ходе выполнения работы были решены следующие задачи:

1. проведен анализ требований и обзор существующих решений в области финансовых платформ и сервисов инвестиционной аналитики;
2. спроектирована и реализована архитектура клиентской и серверной частей приложения;
3. разработан интерфейс для просмотра данных по публичным компаниям, сделкам и портфелю пользователя;
4. реализовано взаимодействие с базой данных и внешним API финансовых инструментов;
5. настроена система аутентификации и сессионного доступа пользователей;
6. обеспечено структурированное хранение и обработка данных в рамках REST-архитектуры.

Достигнутый результат подтверждает возможность реализации полного цикла веб-разработки: от анализа предметной области и проектирования архитектуры до программной реализации и интеграции внешних сервисов. Разработанная в рамках выпускной квалификационной работы платформа может быть

использована как основа для дальнейшего расширения функциональности и практического применения в области инвестиционной аналитики.

Ссылки на работу

1. Figma: <https://www.figma.com/design/iHUIId9cnmlaFMtGI5tyuLP/Untitled?node-id=0-1&t=Kdc22fYy6CcnGhNk-1>
2. Github-репозиторий: <https://github.com/daniltsapaev2003/Ledgerlyproject.git>

Библиографический список

1. Fielding, R. T. Architectural styles and the design of network-based software architectures : dissertation / R. T. Fielding. – Irvine : University of California
2. Silberschatz, A. Database System Concepts / A. Silberschatz, H. F. Korth, S. Sudarshan. – 7th ed. – New York : McGraw-Hill, 2019. – 1376 p.
3. Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Boston : Prentice Hall, 2017. – 432 p.
4. Gamma, E. Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma, R. Helm, R. Johnson, J. Vlissides. – Boston : Addison-Wesley, 1994. – 395 p.
5. Bogner, J. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub / J. Bogner, M. Merkel // Proceedings of the 19th International Conference on Mining Software Repositories. – 2022. – P. 658–669.
6. React Documentation [Электронный ресурс]. – Режим доступа: <https://react.dev/>
7. Vite Documentation [Электронный ресурс]. – Режим доступа: <https://vite.dev/guide/>
8. TypeScript: JavaScript With Syntax For Types [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/>
9. Node.js Documentation [Электронный ресурс]. – Режим доступа: <https://nodejs.org/en/docs>
10. Express – Node.js web application framework [Электронный ресурс]. – Режим доступа: <https://expressjs.com/>
11. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/>
12. Drizzle ORM Documentation [Электронный ресурс]. – Режим доступа: <https://orm.drizzle.team/>
13. Passport.js Documentation [Электронный ресурс]. – Режим доступа: <https://www.passportjs.org/docs/>
14. express-session Documentation [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/resources/middleware/session.html>
15. TanStack Query Documentation [Электронный ресурс]. – Режим доступа: <https://tanstack.com/query/latest>
16. Tailwind CSS Documentation [Электронный ресурс]. – Режим доступа: <https://tailwindcss.com/docs/installation>
17. shadcn/ui Documentation [Электронный ресурс]. – Режим доступа: <https://ui.shadcn.com/>
18. Wouter Documentation [Электронный ресурс]. – Режим доступа: <https://github.com/molefrog/wouter>
19. Git Documentation [Электронный ресурс]. – Режим доступа: <https://git-scm.com/doc>
20. GitHub Docs [Электронный ресурс]. – Режим доступа: <https://docs.github.com/>
21. Figma [Электронный ресурс]. – Режим доступа: <https://www.figma.com/>
22. Tinkoff Invest API [Электронный ресурс]. – Режим доступа: <https://developer.tbank.ru/invest/>
23. TradingView [Электронный ресурс]. – Режим доступа: <https://www.tradingview.com/>
24. Yahoo Finance [Электронный ресурс]. – Режим доступа: <https://finance.yahoo.com/>
25. Investing.com [Электронный ресурс]. – Режим доступа: <https://www.investing.com/>

